Documento de Configuração — Acesso Externo SAAC ERP

Projeto: SAAC ERP — Igreja Evangélica Assembleia de Deus Madureira — Cosmópolis/SP
Data: 05 de novembro de 2026
Autor: Configuração realizada com auxílio do Code Copilot
Ambiente: Spring Boot (microserviços) + Angular + Spring Security

---

1. Visão Geral

Este documento descreve as configurações necessárias para permitir o acesso externo ao sistema SAAC ERP a partir de uma rede fora da estrutura local (ex.: celular 4G/5G ou outra rede).

O sistema é composto por 4 aplicações independentes:

Aplicação	Porta	Descrição
Frontend Angular	`4200`	Interface web acessada pelos usuários
Authorization Server	`8083`	Servidor de autenticação e emissão de tokens JWT
Secretaria (API)	`8081`	Módulo de Secretaria
Tesouraria (API)	`8082`	Módulo de Tesouraria

---

2. Pré-requisitos

Antes de realizar as configurações, verifique se você possui:

- Acesso administrativo ao roteador da rede.
- IP WAN (público) da conexão de internet. Para descobrir, acesse: https://meuip.com.br

- Máquina servidor com IP fixo dentro da rede interna (recomendado).
- Firewall liberando as portas utilizadas (`4200` , `8081` , `8082` , `8083`).
- APIs configuradas para escutar em `0.0.0.0` .

> ⚠️ **Atenção:** Se a conexão de internet usar **CGNAT** (comum em links 4G/5G residenciais), o redirecionamento de portas pode não funcionar mesmo seguindo este documento.

---

3. Configuração das APIs (Backend)

Todas as APIs (Authorization Server, Secretaria e Tesouraria) devem ter:

1. CORS configurado no `SecurityFilterChain` .
2. Bean `CorsConfigurationSource` com as origens permitidas.
3. Propriedade `server.address=0.0.0.0` .

3.1. Authorization Server — `saac_security`

Arquivo: `SecurityConfig.java`

```
package br.com.adbrascosmopolis.saac_security.config;

import
br.com.adbrascosmopolis.saac_security.security.CustomAuthenticationEntryPoint;
import br.com.adbrascosmopolis.saac_security.security.JwtAuthFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import
org.springframework.security.config.annotation.authentication.configuration.Authenti
cationConfiguration;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodS
ecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

```
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity
;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFi
lter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final CustomAuthenticationEntryPoint authenticationEntryPoint;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter,
                          CustomAuthenticationEntryPoint authenticationEntryPoint) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.authenticationEntryPoint = authenticationEntryPoint;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider(UserDetailsService
userDetailsService,
                                                            PasswordEncoder passwordEncoder) {
```

```
        DaoAuthenticationProvider provider = new
DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder);
        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
                .cors(cors -> cors.configurationSource(corsConfigurationSource()))
                .csrf(csrf -> csrf.disable())
                .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .exceptionHandling(ex ->
ex.authenticationEntryPoint(authenticationEntryPoint))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/login", "/api/auth/refresh").permitAll()
                        .anyRequest().authenticated()
                )
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(List.of("http://201.130.23.160:4200"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH",
"DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type",
"X-Requested-With"));
        configuration.setExposedHeaders(List.of("Authorization"));
        configuration.setAllowCredentials(true);
```

```
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

Arquivo: `application.properties`

```
server.port=8083
server.address=0.0.0.0
```

3.2. Tesouraria — `saac_tesouraria`

Arquivo: `SecurityConfig.java`

```
package br.com.adbrascosmopolis.saac_tesouraria.config;

import br.com.adbrascosmopolis.saac_tesouraria.security.JwtAuthFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodS
ecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigur
er;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFi
lter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
```

```
import java.util.Arrays;
import java.util.List;

@Configuration
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .cors(cors -> cors.configurationSource(corsConfigurationSource()))
                .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                        .requestMatchers("/actuator/health").permitAll()
                        .anyRequest().authenticated()
                )
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        configuration.setAllowedOrigins(List.of(
                "http://localhost:4200",
                "http://10.0.0.76:4200",
                "http://201.130.23.160:4200"
        ));

        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH",
```

```
"DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setExposedHeaders(List.of("Authorization"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);

        return source;
    }
}
```

Arquivo: `application.properties`

```
server.port=8082
server.address=0.0.0.0
```

3.3. Secretaria — `saac_secretaria`

Arquivo: `SecurityConfig.java`

```
package br.com.adbrascosmopolis.saac_secretaria.config;

import br.com.adbrascosmopolis.saac_secretaria.security.JwtAuthFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodS
ecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigur
er;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import
```

```
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFi
lter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .cors(cors -> cors.configurationSource(corsConfigurationSource()))
                .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                        .requestMatchers("/actuator/health").permitAll()
                        .anyRequest().authenticated()
                )
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        configuration.setAllowedOrigins(List.of(
```

```
                "http://localhost:4200",
                "http://10.0.0.76:4200",
                "http://201.130.23.160:4200"
        ));

        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH",
"DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setExposedHeaders(List.of("Authorization"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);

        return source;
    }
}
```

Arquivo: `application.properties`

```
server.port=8081
server.address=0.0.0.0
```

---

4. Configuração do Frontend Angular

4.1. Ambiente

No arquivo `src/environments/environment.ts`, configure as URLs das APIs apontando para o IP WAN:

```
export const environment = {
  production: false,
  apis: {
    security: 'http://201.130.23.160:8083',
    secretaria: 'http://201.130.23.160:8081',
    tesouraria: 'http://201.130.23.160:8082'
```

```
  }
};
```

4.2. Iniciar o Angular para acesso externo

O comando padrão `ng serve` só libera acesso local. Para permitir conexões externas, use:

```
ng serve --host 0.0.0.0 --port 4200
```

Ou, se estiver usando um script no `package.json`:

```
"scripts": {
  "start:external": "ng serve --host 0.0.0.0 --port 4200"
}
```

Execute com:

```
npm run start:external
```

---

5. Redirecionamento de Portas no Roteador

Acesse o painel administrativo do roteador (geralmente `192.168.0.1` ou `192.168.1.1`) e configure o redirecionamento de portas para o IP fixo da máquina servidor (`10.0.0.76` no nosso caso):

Porta Externa	Porta Interna	IP Destino	Protocolo	Aplicação
4200	4200	10.0.76	TCP	Frontend Angular
8081	8081	10.0.76	TCP	Secretaria
8082	8082	10.0.76	TCP	Tesouraria
8083	8083	10.0.76	TCP	Authorization Server

> ⚠️ **Importante:** mantenha o IP da máquina servidor fixo no roteador (reserva DHCP) para evitar que o redirecionamento pare de funcionar.

---

6. Configuração do Firewall da Máquina Servidor

No Windows, certifique-se de liberar as portas `4200`, `8081`, `8082` e `8083` para conexões de entrada.

No PowerShell como administrador:

```
New-NetFirewallRule -DisplayName "SAAC Angular" -Direction Inbound -LocalPort 4200 -Protocol TCP -Action Allow
New-NetFirewallRule -DisplayName "SAAC Secretaria" -Direction Inbound -LocalPort 8081 -Protocol TCP -Action Allow
New-NetFirewallRule -DisplayName "SAAC Tesouraria" -Direction Inbound -LocalPort 8082 -Protocol TCP -Action Allow
New-NetFirewallRule -DisplayName "SAAC Authorization Server" -Direction Inbound -LocalPort 8083 -Protocol TCP -Action Allow
```

---

7. Fluxo de Teste

1. Inicie as 3 APIs na máquina servidor.
2. Inicie o Angular com `--host 0.0.0.0`.
3. Na própria máquina servidor, acesse: `http://201.130.23.160:4200/login`
4. Faça login com um usuário válido.
5. Verifique se os módulos de Secretaria e Tesouraria carregam corretamente.
6. Em seguida, teste pelo celular usando a rede 4G/5G:
   `http://201.130.23.160:4200/login`

---

8. Cuidados de Segurança

- Nunca exponha o backend diretamente sem autenticação.
- Prefira HTTPS em produção (certificado SSL).
- Evite usar `*` em `AllowedOrigins` quando `AllowCredentials` for `true`.

- Mantenha senhas fortes e roteie periodicamente os segredos JWT.
- Monitore logs de acesso suspeitos.
- Considere utilizar um domínio + proxy reverso (ex.: Nginx ou Cloudflare) em vez de IP direto.

---

9. Problemas Comuns

Problema	Possível Causa	Solução
Tela de login não carrega	Roteador ou firewall bloqueando porta `4200`	Verificar redirecionamento e firewall
Login funciona, mas APIs dão erro	CORS configurado incorretamente	Revisar `AllowedOrigins` em cada API
Funciona no PC, mas não no celular	CGNAT da operadora	Solicitar IP público fixo ao provedor ou usar VPN/Tunnel
APIs dando 401 diretamente no navegador	Comportamento esperado	APIs exigem token JWT. Teste sempre pelo frontend

---

10. Observações Finais

- Sempre que o IP WAN mudar, atualize as URLs no `environment.ts` do Angular e nas configurações de CORS das APIs.
- Para ambientes de produção, recomenda-se utilizar HTTPS com certificado válido.
- Este documento reflete a configuração vigente em novembro de 2026.
